

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T14:01:22.0607703Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1 (BENCHMARK_WIDTH=600 B
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Carte benchmark	600 × 600 (densité 0.25)
Profil CPU	cpu-20260924-140122062.prof
Profil mémoire	memory-20260924-140122062.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	12.40 ms	12395032	3	0

Détail hyperfine

Métrique	Valeur
Moyenne	5.6847 s
Écart-type	n/a
Médiane	5.6847 s
Temps utilisateur	34.0862 s
Temps système	1.1006 s
Min	5.6847 s
Max	5.6847 s
Runs mesurés	1
Coefficient de variation	n/a
Pic WorkingSet commande Go	26.62 MiB
Pic mémoire privée commande Go	60.64 MiB

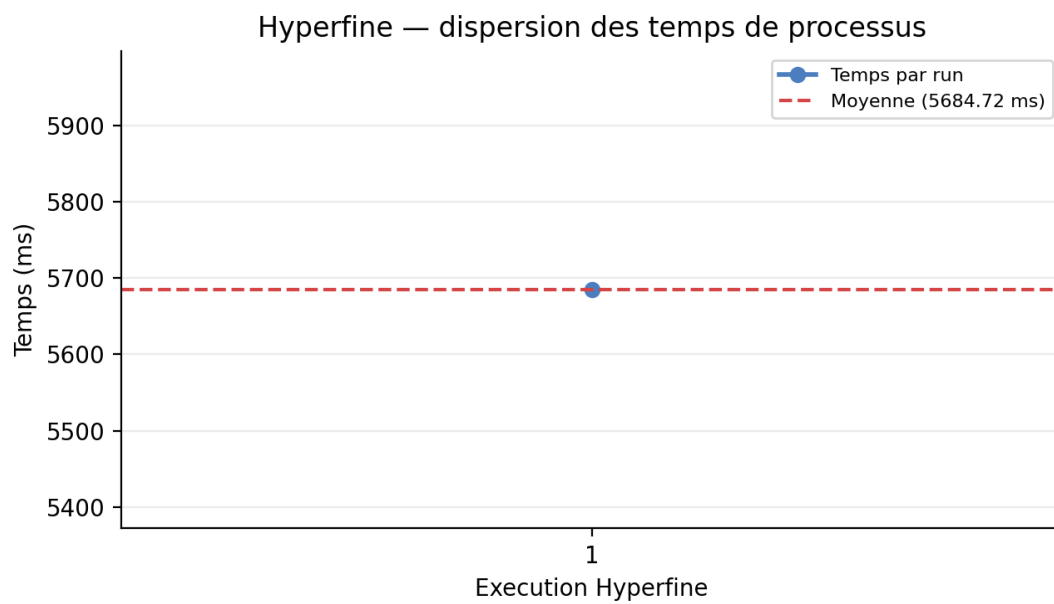


Figure — Temps de chaque exécution Hyperfine et moyenne.

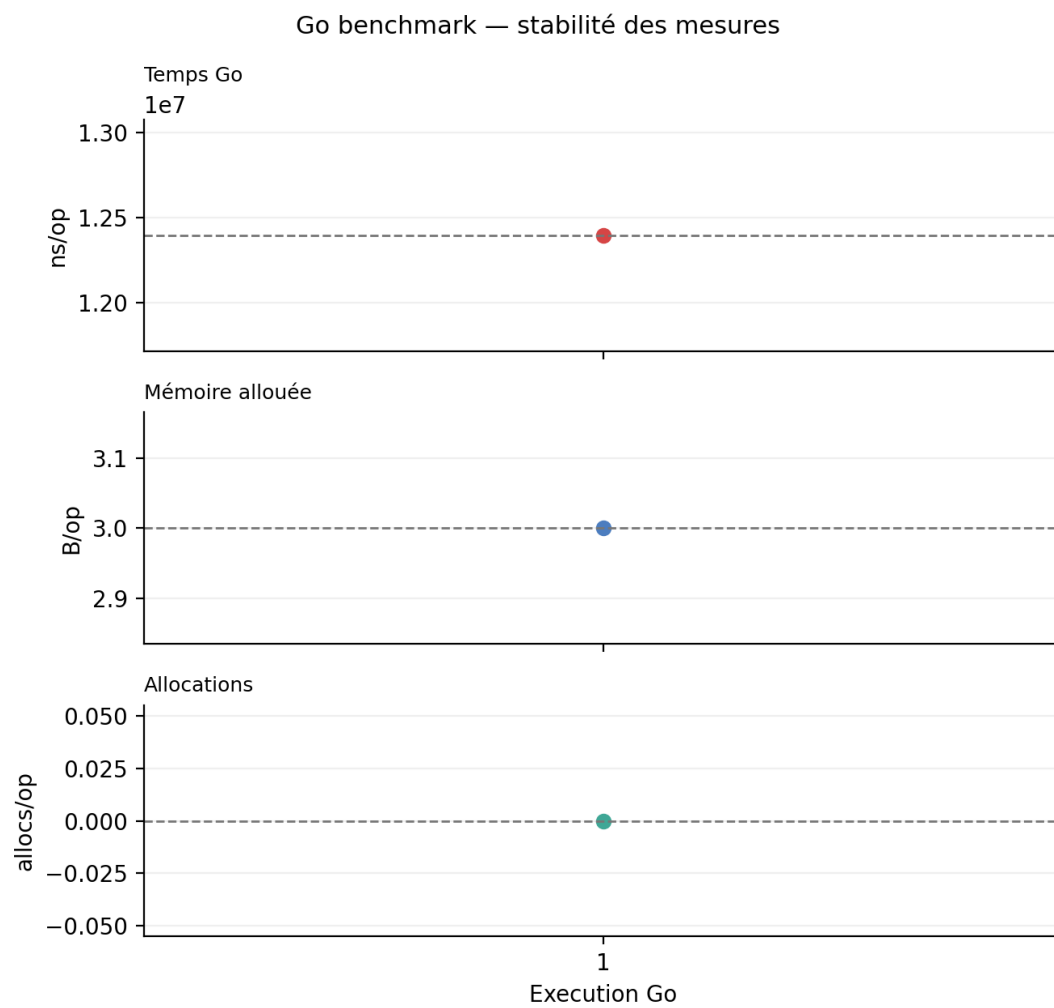


Figure — Dispersion des mesures Go : temps, mémoire allouée et allocations.

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	38.31s
Mémoire allouée (pprof)	79.20MB
Heap vivant (pprof)	7177.29kB
RAM physique totale	32061 MiB
RAM physique disponible	12974 MiB
RAM physique utilisée	19087 MiB
Mémoire par opération	3 B/op

Garbage collector Go

Cycles GC	66
Temps GC cumulé	79.210 ms
Plus longue pause	3.960 ms
Temps CPU GC cumulé	356.480 ms
Objectif mémoire maximal	97.00 MB

La RAM physique est l'état du système au lancement du benchmark. La mémoire pprof mesure le heap Go échantillonné (alloué ou encore vivant), pas toute la mémoire RSS du processus. Hyperfine peut fournir le RSS quand le backend système le supporte ; une valeur nulle est donc signalée comme indisponible.

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **38.31s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	36.16s	94.39%	36.75s	95.93%
jeu-de-la-vie/game.(*Board).Step	0.66s	1.72%	1.20s	3.13%
runtime.asyncPreempt	0.59s	1.54%	0.59s	1.54%
jeu-de-la-vie/game.startCandidatePool.func1	0.27s	0.70%	37.04s	96.68%
math/rand.(*Rand).Int63 (inline)	0.25s	0.65%	0.37s	0.97%
math/rand.(*Rand).Float64 (inline)	0.12s	0.31%	0.49s	1.28%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	1.22s	3.18%
testing.(*B).launch	0	0.00%	1.22s	3.18%

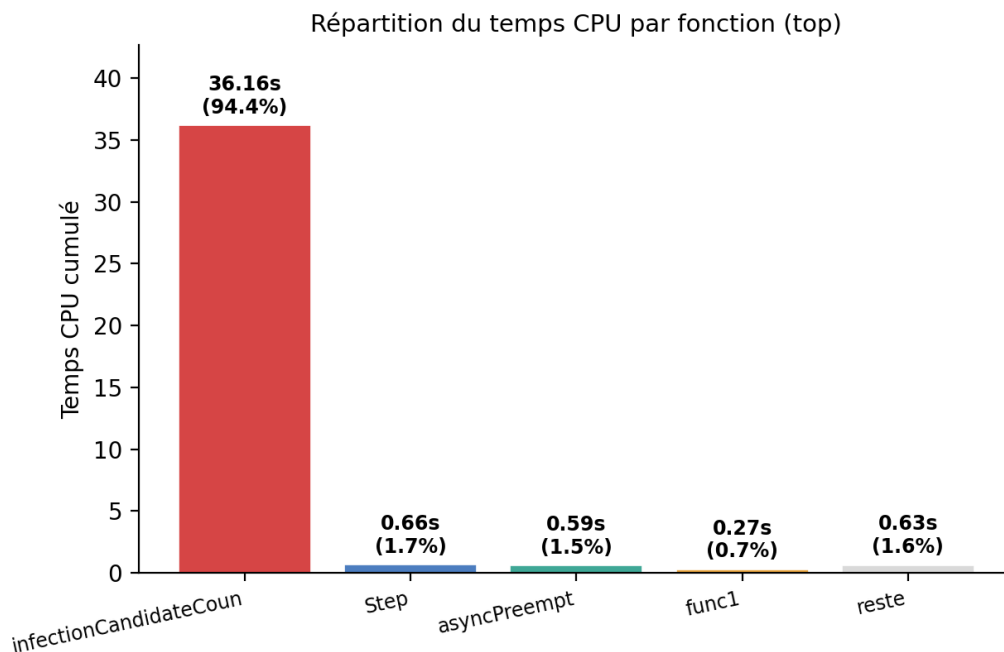


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

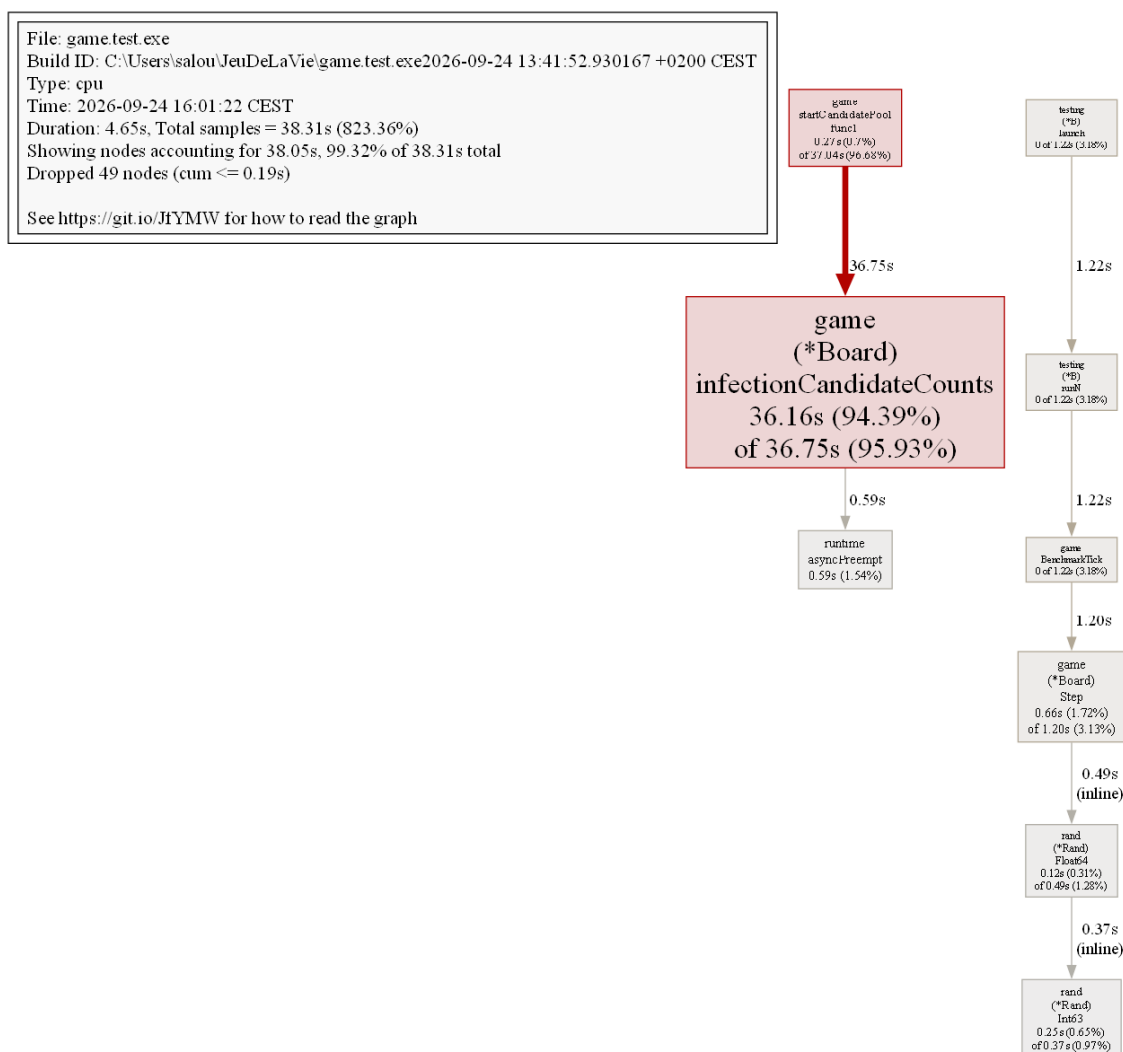


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **94.4%** du temps CPU total (flat), pour **95.9%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

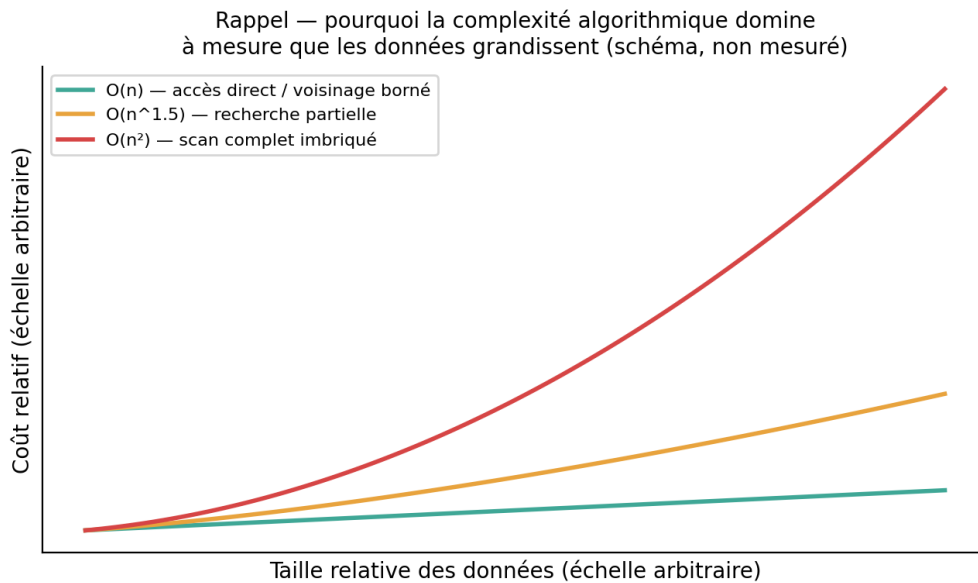


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **79.20MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard	53.16MB	67.12%	53.16MB	67.12%
jeu-de-la-vie/game.RandomBoard	13.81MB	17.43%	66.97MB	84.55%
runtime.mallocgc	4.51MB	5.69%	4.51MB	5.69%
jeu-de-la-vie/game.startCandidatePool.func1	2MB	2.53%	2MB	2.53%
compress/flate.(*compressor).init	1.17MB	1.47%	1.17MB	1.47%
runtime/pprof.StartCPUProfile	1.16MB	1.46%	1.16MB	1.46%
compress/flate.NewWriter (inline)	0.88MB	1.11%	2.05MB	2.59%
jeu-de-la-vie/game.buildCandidateOffsets (inli	0.52MB	0.66%	0.52MB	0.66%

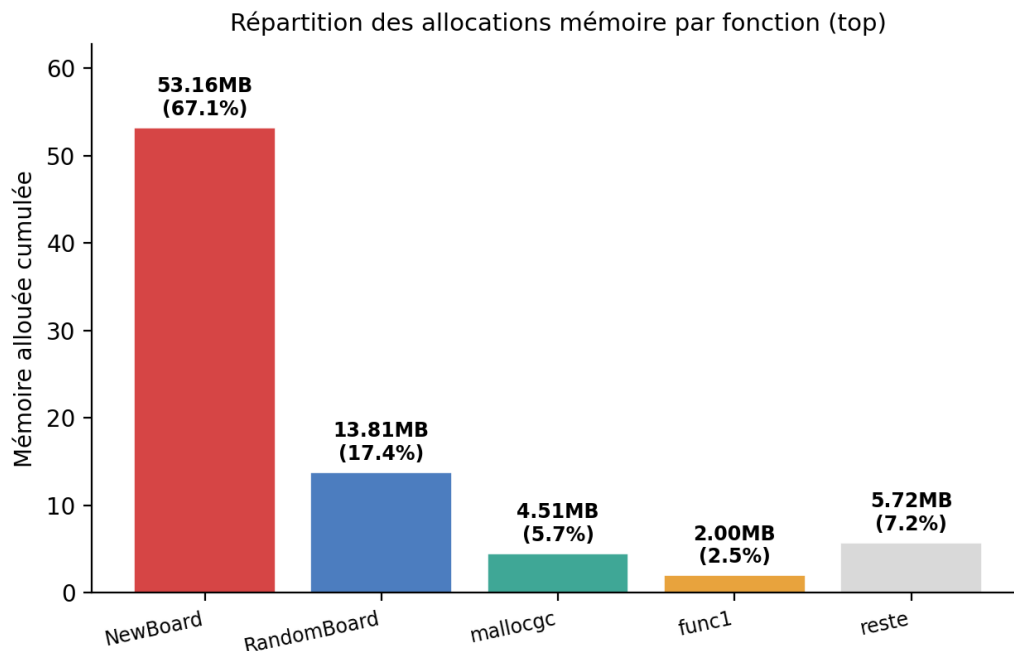


Figure — Répartition des allocations mémoire par fonction.

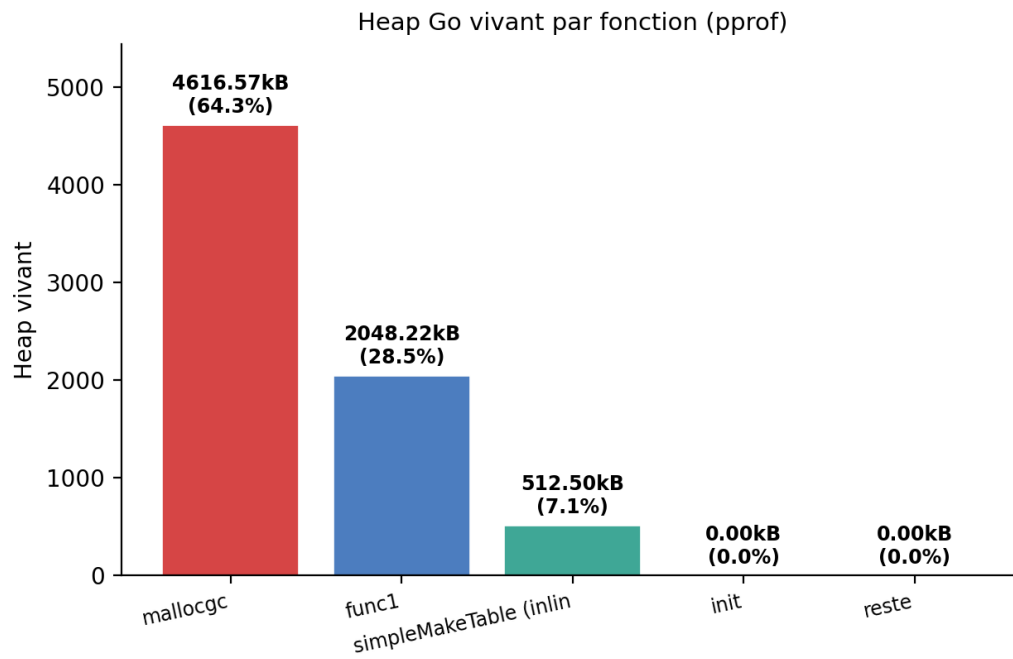


Figure — Heap Go encore vivat par fonction (pprof -inuse_space).

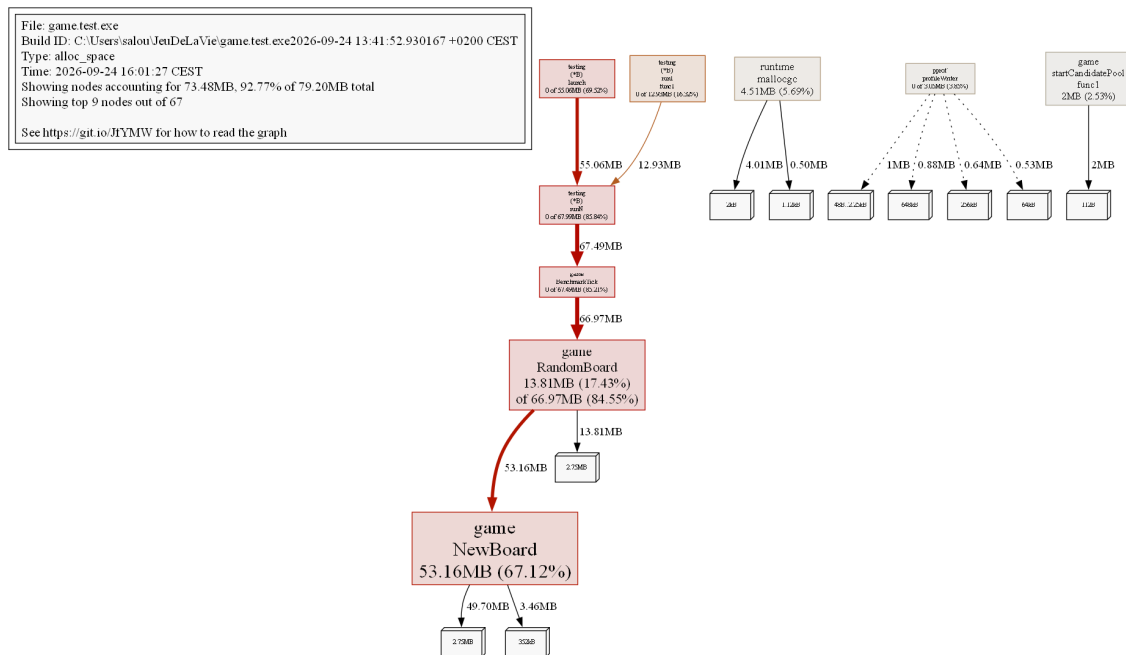


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- Densité de la carte : 0.25 ; seed : 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 16:01:46 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.